

abr. 22, 2026

Integrações Síncronas - HTTP

convidado Bruno Joaquim Cassia Dobler Fabio Diniz Francisco Schneider
Gabriel Severo Gabriel kohlrausch Ruan Rodrigues
aline.bianchini@t-systems.com ana.santos@t-systems.com
analista.kuehn@outlook.com anderson.hilario@t-systems.com
anderson.perim@t-systems.com andersongabriel_maria@hotmail.com
anselmogabriel421@gmail.com bmf.pedro@gmail.com
bruna.nery@t-systems.com bruno.destro@t-systems.com
caique.rodrigues@t-systems.com d.silva@t-systems.com
danilosilva2017@hotmail.com desenvolvedor@tidsoftware.com.br
douglas.nishiyama@t-systems.com dpsnbsb79@gmail.com
eduardo.kubota@t-systems.com eudy.amparo@t-systems.com
fabio.marinelli@t-systems.com farley.t.i@hotmail.com
felipeneves089@gmail.com gustavo.ferrazfontes@gmail.com
hewerton.souza@t-systems.com i.parmigiani@t-systems.com
igor.romulo.santos@gmail.com jeanderson.silva@t-systems.com
jl.oliveir@gmail.com jonathan.jimenez@iatec.com karinetrevisann@gmail.com
l.regioli@outlook.com leandro.piqueira@t-systems.com
lucas.p.oliveira@outlook.pt lxsilvareis@gmail.com
marcelohcordeiro@gmail.com marcio@almob.com.br
marcos.ammon@gmail.com moraesguga@gmail.com n.segeti@t-systems.com
nadson.dr11@gmail.com Paulo Scatena rafael.dias@t-systems.com
regis.maioli@t-systems.com reinaldo.rodrigues@t-systems.com
renan-silva@t-systems.com renan.baptista.cabral@gmail.com
rodrigo_7sch@hotmail.com tatidornel@gmail.com
thiago.fernandes@t-systems.com victor.prospero@t-systems.com
victor.santos@t-systems.com victor.souza@t-systems.com
vinicius.bonicio@t-systems.com webdeveloperrm@gmail.com
wesley.boccaletto@t-systems.com wsilva@t-systems.com
fabiano.felgas@gmail.com

Anexos  Integrações Síncronas - HTTP○○○○○

Resumo

A aula explorou otimização de sistemas .NET via assincronismo, gerenciamento de threads e técnicas de alta performance.

Fundamentos de Threads e Assincronismo

O treinamento detalhou o uso de Thread Pool, a distinção entre processos síncronos e assíncronos, além da importância de evitar o travamento de recursos com o uso correto de `async` e `await`. A decisão principal foi priorizar a reutilização de threads e evitar o uso de `Thread.Sleep` em favor de `Task.Delay` para otimizar o hardware.

Estratégias de Performance e Integração

Foram apresentadas técnicas avançadas como `IHttpClientFactory` para mitigar o esgotamento de sockets e o uso da biblioteca Polly para resiliência. O foco recaiu sobre o gerenciamento de tarefas paralelas e a utilização eficiente de memória com `Span` e `ValueTask`.

Otimização de Recursos e Arquitetura

A sessão final abordou padrões como Produtor-Consumidor com Channels e Dataflow para fluxos complexos, enfatizando a importância de testes de carga. Encerrou-se com a recomendação de utilizar `Value Objects` para melhorar a expressividade e a performance do código.

Próximas etapas

- ☐ [Fabio Diniz] Compartilhar Links: Publicar links mencionados no grupo WhatsApp. Garantir que os materiais de referência sejam acessíveis.

Detalhes

- **Boas-vindas e Introdução ao Tópico:** Fabio Diniz iniciou a sessão dando as boas-vindas e introduzindo o tópico da aula, que seria sobre integrações síncronas e as formas de realizá-las. Eles solicitaram que as dúvidas fossem deixadas para o final da apresentação, mas permitiram intervenções para questões mais complexas ([00:00:00](#)).
- **Fundamentos de Processos Síncronos:** Foi apresentado um exemplo de processo síncrono utilizando a compra e venda de um imóvel, que envolve um comprador, banco, vendedor e cartório. Nesse exemplo, a etapa de análise de crédito pelo banco deve ser concluída antes que as etapas seguintes, como a realização da operação e o registro do bem no cartório de imóveis, possam ocorrer ([00:03:52](#)).
- **Exemplos Práticos de Processos Síncronos:** Alguns exemplos práticos de processos síncronos foram citados, incluindo o processamento de pagamento em compras online, onde o despacho do pedido só ocorre após a confirmação da transação. Outros exemplos são a autenticação de usuários e a reserva de passagens aéreas ou consultas, onde é necessário aguardar a validação de credenciais ou a confirmação de disponibilidade ([00:05:15](#)).
- **Conceito de Threads, Núcleos e Hardware:** O instrutor abordou o conceito de *threads* e núcleos, explicando como eles impactam o pensamento de sistemas, e mencionou que a maneira como as linguagens de programação implementam certas funcionalidades é determinada pela forma como o hardware opera. Em um exemplo de *hardware*, observou-se que um sistema pode ter quatro núcleos e quatro processadores lógicos, enquanto outro pode ter seis núcleos físicos, mas 12 processadores lógicos através do *hyper threading* ou SMT, o que é uma abstração fornecida pelo hardware ao sistema operacional ([00:06:28](#)).
- **Modelos de Threads e Controle pelo Sistema Operacional (SO):** Existem modelos de *threads*, incluindo as *threads* de núcleo, que são as que o sistema operacional gerencia. Quando um software é iniciado, o SO aloca *threads* (linhas de execução) para o software trabalhar, controlando e agendando quando cada processo será executado ([00:09:11](#)). O SO utiliza um escalonador de processos para conceder tempo de CPU a cada processo em execução ([00:10:19](#)).
- **Troca de Contexto e Uso de Recursos:** A troca de contexto entre *threads* é dispendiosa e envolve o armazenamento de dados nos registradores do hardware, formando um contexto ([00:11:33](#)). É essencial fazer um bom uso dos recursos, pois eles são limitados e escassos ([00:10:19](#)) ([00:16:17](#)). O SO

e o hardware utilizam níveis de memória **cache** (L1, L2, L3) para otimizar as trocas de contexto, mantendo dados próximos ao processador para reduzir o custo ([00:11:33](#)).

- **Threads de Usuário e Híbridas:** Além das **threads** de núcleo gerenciadas pelo SO, existem as **threads** de usuário, que são gerenciadas por uma biblioteca ou implementação proprietária dentro do processo ([00:13:52](#)). Também existem **threads** híbridas, que combinam a flexibilidade do nível de usuário com a robustez do nível de núcleo, fazendo o mapeamento entre elas. O .NET utiliza as **threads** de núcleo, que são cedidas e gerenciadas pelo SO ([00:15:09](#)).
- **Conceito de Thread Pool no .NET:** O .NET implementa o conceito de **Thread Pool** (piscina de **threads**), uma das vantagens do C\#ARP, para gerenciar as **threads** obtidas do SO e reutilizá-las em várias tarefas. Isso evita o excesso de criação de **threads** e o **overhead** associado, o que é um esforço para otimizar o uso dos recursos escassos do hardware ([00:16:17](#)).
- **Chamadas Síncronas vs. Assíncronas:** Uma chamada síncrona devolve o controle ao chamador somente após a conclusão da operação. Em contraste, uma chamada assíncrona retorna o controle imediatamente, permitindo que o chamador continue a execução enquanto a operação é realizada em paralelo ([00:20:00](#)). Ao utilizar ``Task.Run`` no .NET, o sistema solicitará uma **thread** do **Thread Pool** ([00:21:11](#)).
- **Análise de Chamadas Assíncronas e Await:** Em chamadas assíncronas com ``Task.Run``, o sistema retorna imediatamente o controle, não se preocupando com a execução da tarefa em si ([00:22:25](#)). A utilização de ``await`` faz com que o chamador aguarde a conclusão da tarefa, mas para evitar que a **thread** principal fique presa, o .NET a disponibiliza de volta no **Thread Pool** para outras tarefas ([00:24:46](#)). O DNET faz o reaproveitamento da **thread** ([00:26:59](#)).
- **Convenções e Riscos em Métodos Assíncronos:** Por convenção no .NET, métodos que terminam com a assinatura **Async** retornam uma ``Task``. Foi enfatizada a cautela com o uso de ``void`` em métodos assíncronos, o que é conhecido como **fire and forget** ([00:28:08](#)). Se uma exceção ocorrer em um método **void** assíncrono, ela pode ser de difícil captura, apesar de haver usos pontuais para essa prática, como em **logging** ([00:29:11](#)).
- **Retornando Tasks e Offload de Carga:** Quando um método retorna uma ``Task``, a responsabilidade de decidir se aguarda ou não fica com quem chamou o método. Os exemplos de ``Task.Run`` ilustram o **offload** de carga, onde uma nova **thread** é alocada do **Thread Pool** para executar o trabalho

em paralelo ([00:30:08](#)). A utilização de ``await`` aqui também libera a `*thread*` chamadora, permitindo seu reaproveitamento ([00:31:16](#)).

- **Distinção entre Processo Síncrono e Chamada Síncrona:** Em resposta a uma pergunta de Felipe Neves, Fabio Diniz esclareceu a diferença entre um processo de negócio síncrono e chamadas síncronas ou assíncronas dentro do código. O processo de negócio pode ser síncrono (exigindo um resultado antes de prosseguir), mas pode conter chamadas assíncronas internamente, e ``Task.Run`` sempre solicitará uma nova `*thread*` ([00:33:21](#)).
- **Task.Delay vs. Thread.Sleep:** Foi destacada a importância de não usar ``Thread.Sleep``, pois isso paralisa a `*thread*`, impedindo seu reaproveitamento e uso do recurso ([00:40:41](#)). O ``Task.Delay`` é o método preferível, pois notifica o DNET de que a `*task*` entrará em atraso, permitindo que a `*thread*` seja devolvida ao `*Thread Pool*` para uso em outras tarefas ([00:39:20](#)).
- **CPU Bound e IO Bound:** As operações `*CPU Bound*` são limitadas pelo processador, exigindo muito processamento (cálculos complexos, compressão de dados, renderização). As operações `*IO Bound*` dependem de entrada e saída de dados (acesso a rede, armazenamento). O instrutor forneceu exemplos de `*CPU Bound*` que fazem `*offload*` de trabalho para uma `*thread*` no `*Thread Pool*` ([00:41:54](#)).
- **Uso da Assincronia e Processos IO Bound:** Foi questionado se a assincronia era mais adequada apenas para processos `*IO Bound*` ([00:49:08](#)). Fabio Diniz reforçou que não há obrigatoriedade de vincular assincronismo a `*IO Bound*` ou `*CPU Bound*`, dependendo da forma como o código é programado ([00:49:59](#)). O `*hardware*` é assíncrono por natureza, e a decisão de usar sincronismo ou assincronismo depende da necessidade do programador ([00:51:07](#)).
- **Exemplo de IO Bound com Assincronismo:** Em um exemplo de leitura de arquivo (`*IO Bound*`), a chamada é assíncrona (``await``), mas o sistema aguarda a conclusão da leitura para continuar, garantindo que o conteúdo esteja disponível para retorno. O DNET disponibiliza a `*thread*` chamadora no `*Thread Pool*` enquanto a leitura é realizada em paralelo ([00:52:11](#)).
- **Combinação de IO Bound e CPU Bound:** Um exemplo final demonstrou como unir `*IO Bound*` (leitura de arquivo) e `*CPU Bound*` (processamento dos dados lidos). O código realiza a leitura assíncrona e, em seguida, executa o processamento em paralelo, utilizando o ``await`` para garantir que o processamento do conteúdo lido seja concluído antes de seguir ([00:54:14](#)).
- **Reflexão sobre a Otimização de Recursos:** Foi enfatizado que o assincronismo nem sempre é a melhor opção, pois o uso excessivo de

chamadas assíncronas pode sobrecarregar recursos escassos, como disco ou rede, acessando-os em paralelo ([00:55:15](#)). É crucial usar os recursos da máquina da melhor forma possível ([00:56:28](#)).

- **Assincronismo e Paralelismo:** Foi feita uma distinção importante sobre a ordem de execução do código assíncrono ([00:57:33](#)). No primeiro caso, com ``await`` antes da segunda chamada, a segunda `*task*` (leitura do `*file 2*`) só será iniciada após a conclusão da primeira `*task*` (leitura do `*file 1*`) ([00:58:45](#)). Em contraste, ao iniciar duas `*tasks*` de leitura sem o ``await`` imediato, ambas as operações serão iniciadas e processadas em paralelo ([00:57:33](#)).
- **Processamento Assíncrono e Paralelo de Tarefas de Leitura:** Fabio Diniz demonstrou o processo de execução de tarefas (tasks) de leitura de forma assíncrona, onde uma tarefa é instanciada e, em seguida, a próxima é iniciada imediatamente, permitindo que as leituras ocorram em paralelo ([01:00:45](#)). Ele enfatizou que o uso do comando ``wait`` na primeira tarefa garante que o fluxo do programa não continue até que essa tarefa específica tenha sido concluída, mesmo que outras tarefas em paralelo possam terminar antes ([01:01:37](#)). A distinção entre assíncrono (iniciar e obter controle de volta) e paralelo (execução simultânea) foi destacada, mencionando que a leitura em paralelo na mesma unidade de disco pode afetar o `*throughput*` total da unidade ([01:03:26](#)).
- **Comparação de Performance em Diferentes Cenários de Leitura:** Karine Trevisan perguntou qual abordagem seria melhor em termos de performance, especialmente sem conhecimento prévio do tamanho dos arquivos. Fabio Diniz explicou que, quando uma tarefa não depende do resultado da anterior, a execução paralela das leituras sem um ``await`` imediato pode oferecer ganho ([01:02:29](#)). Entretanto, eles concordaram que a leitura em paralelo na mesma unidade de armazenamento pode sobrecarregar essa unidade, limitando a taxa de leitura total, já que a unidade tem um `*throughput*` máximo ([01:03:26](#)).
- ****Gerenciamento de Múltiplas Tarefas com `WhenAll`**:** Foi apresentada a técnica ``WhenAll`` para gerenciar múltiplas operações de I/O, como leitura de dois arquivos e uma chamada de rede ([01:05:22](#)). Essa técnica cria uma tarefa que aguarda o término de todas as tarefas fornecidas (arquivos e rede) antes de prosseguir, garantindo que o fluxo não avance até a finalização de todas. O uso do ``await`` no ``WhenAll`` garante o bloqueio do fluxo até que todas as tarefas sejam concluídas, não importando a ordem em que elas finalizam ([01:06:32](#)).

- ****Diferença entre `WhenAll` e `Task.WaitAll`****: Fabio Diniz explicou que o `WhenAll` é usado em contextos assíncronos (`async/await`) para aguardar a finalização de múltiplas tarefas sem bloquear o `*thread*` de execução, enquanto a forma `Task.WaitAll` é bloqueante, significando que o `*thread*` que a chama fica parado aguardando o resultado ([01:11:29](#)). Ele ressaltou que bloqueio significa que o `*thread*` não é liberado de volta para o `*pool*`, mantendo o recurso travado ([01:12:36](#)). Felipe Neves confirmou que o `WhenAll` apenas espera o que já foi iniciado, não sendo responsável por trigar as tarefas ([01:08:30](#)).
- ****Gerenciamento de Múltiplas Tarefas com `WhenAny`****: A técnica `WhenAny` foi introduzida como uma forma de processar tarefas à medida que são concluídas, sem a necessidade de esperar que todas finalizem. Essa abordagem é útil quando a ordem de conclusão das tarefas não é importante, permitindo que o sistema processe imediatamente a tarefa que terminar primeiro. O processo envolve um `*loop*` que espera (`await`) por qualquer tarefa na lista para finalizar, remove a tarefa concluída da lista e a processa ([01:09:22](#)).
- ****Integrações via HTTP e Uso de `HttpClient`****: O tópico mudou para integrações HTTP, apresentando o uso da classe nativa `HttpClient` no .NET para realizar chamadas externas ([01:13:44](#)). Foi demonstrado um exemplo simples de chamada `GET` e posterior desserialização do resultado JSON para um objeto tipado ([01:19:40](#)).
- ****Problemas com o Uso Indevido de `HttpClient` (Socket Starvation)****: Foi destacado que o uso incorreto ou intensivo do `HttpClient` pode levar à exaustão de recursos, especificamente `*sockets*`, um problema conhecido como `*socket starvation*`. Embora a classe implemente `IDisposable`, ela não deve ser usada com `using` para liberar `*sockets*`, pois as conexões TCP não são liberadas imediatamente, sendo essa uma limitação do protocolo TCP/IP e não do .NET ([01:20:52](#)) ([01:23:33](#)). Fabio Diniz compartilhou uma experiência onde a aplicação parou de responder porque todas as portas de rede estavam exauridas (`*topadas*`) devido ao uso incorreto do `HttpClient` ([01:24:42](#)).
- ****Estratégias para Evitar o `*Socket Starvation*`****: Em resposta a Wallace Silva, que tem centenas de consultas HTTP por segundo, Fabio Diniz afirmou que não há uma maneira de liberar `*sockets*` mais rapidamente, pois o tempo de espera (`*time wait*`) é uma restrição do protocolo TCP/IP para evitar que pacotes de uma conexão antiga sejam entregues a uma nova conexão na mesma porta ([01:26:58](#)). A solução sugerida é o reaproveitamento do `HttpClient` ([01:28:57](#)).

- ****Uso da `IHttpClientFactory` e Clientes Tipados****: Foi apresentada a `IHttpClientFactory` (introduzida no .NET Core 2.1) como a forma recomendada de gerenciar o ciclo de vida do `HttpClient` e evitar o **socket starvation**. A abordagem de **typed clients** (clientes tipados) permite que o `HttpClient` seja injetado via injeção de dependência, utilizando a mesma instância de forma segura ([01:28:57](#)). Dados de um teste de carga mostraram que o uso da fábrica melhora o desempenho, reduzindo o tempo médio de resposta e aumentando o número de requisições por segundo ([01:31:15](#)).
- **Utilização da Biblioteca Fluency URL (FluUrl)**: A biblioteca `FluUrl` foi introduzida como uma alternativa moderna para chamadas HTTP, oferecendo construção fluente de URLs, serialização/desserialização automática e suporte a resiliência ([01:31:15](#)). Testes de carga indicaram que o `FluUrl` adiciona muito pouco **overhead** e vale a pena pela facilidade de leitura e escrita, pois já é preparado para usar a `IHttpClientFactory` ([01:33:47](#)).
- **Políticas de Resiliência com a Biblioteca Polly**: O Polly é uma biblioteca para criar políticas de resiliência e falha, essenciais em aplicações modernas para manter a funcionalidade e a estabilidade ([01:33:47](#)). As políticas discutidas incluem **Retry** (tentativa de novo com estratégia), **Fallback** (tentar um caminho alternativo) e **Circuit Breaker** (interrupção do fluxo em caso de falha para proteger o sistema de ser sobrecarregado ao retornar) ([01:35:06](#)). Foi feita uma advertência sobre o uso do **Timeout**, especialmente em banco de dados, que existe para proteger o banco de longas conexões ativas ([01:38:12](#)).
- ****Implementação do Padrão *Retry*****: O padrão **Retry** com a biblioteca Polly foi detalhado, permitindo a definição de quais códigos de retorno (erros transitórios) devem acionar uma retentativa. Foi configurada uma política de retentativa máxima de cinco vezes com espera exponencial (**exponent backoff**), onde o tempo de espera entre as tentativas aumenta para evitar sobrecarregar o recurso externo ([01:41:26](#)).
- **Leitura e Escrita de Arquivos: Diferentes Abordagens**: A discussão voltou para a manipulação de arquivos, enfatizando que a escolha do método depende do cenário. O método `ReadToEnd` carrega o arquivo inteiro na memória, o que é simples, mas pode esgotar a memória se o arquivo for muito grande ([01:46:29](#)). O `ReadLine` lê o arquivo linha por linha, o que é mais eficiente para arquivos de texto grandes ([01:47:41](#)).
- **Leitura de Arquivos com Buffers e Threads**: Para arquivos binários ou para ter mais controle, a leitura pode ser feita por **stream** de byte em byte usando um **buffer** (por exemplo, 1024 bytes), o que é comumente usado em compressão/descompressão e operações de rede ([01:50:08](#)). Fabio Diniz

reiterou que nem sempre o assíncrono é a melhor escolha, pois aumenta a complexidade do código e a troca de contexto entre **threads**, adicionando **overhead** (01:51:14).

- **Otimização de Leitura com Paralelismo ForEach:** Para otimizar a leitura de múltiplos arquivos, foi apresentada a função nativa ``Parallel.ForEach``, que distribui automaticamente a carga de trabalho entre os núcleos disponíveis. O ``Parallel.ForEach`` permite configurar o grau máximo de paralelismo e lida com a distribuição da carga (01:55:56). Foi alertado sobre os riscos de **thread safety** e condições de corrida ao acessar variáveis externas em código paralelo (01:59:13).
- : *****Task When All com Particionamento (Batch Size)*****: Foi discutida a técnica de utilizar ``Task.WhenAll`` com particionamento de tarefas em lotes (batches), controlando o número máximo de arquivos ou itens processados por vez. Definir um ``batch size`` permite um controle mais fino sobre a quantidade de tarefas simultâneas, garantindo que no máximo um número definido de itens seja processado por vez, como cinco arquivos.
- * *****Desafios em Ambientes Multitenant e Paralelismo*****: Fábio Diniz compartilhou um exemplo real de um sistema multitenant onde os bancos de dados eram separados logicamente, mas usavam a mesma instância do SQL Server, o que causava problemas de desempenho durante a importação paralela de um grande volume de faturas. A solução encontrada foi implementar estratégias mais ajustadas, como permitir execuções paralelas, mas não no mesmo banco, para evitar travamentos.
- * *****Padrão Produtor-Consumidor com Channel-Based*****: Uma abordagem alternativa para concorrência é o modelo produtor-consumidor usando ``Channel-Based``, onde um produtor adiciona tarefas a um canal e um consumidor as retira para execução. Esta arquitetura oferece boa escalabilidade e gerenciamento automático de "back pressure", embora introduza complexidade adicional e uma latência inicial, pois o consumidor espera que o produtor prepare os itens.
- * *****Dataflow Block (Transform Block e Action Block)*****: Foi apresentado o conceito de ``Dataflow Block`` para compor fluxos de trabalho mais complexos, permitindo definir o paralelismo e encadear blocos, como o ``Transform Block`` (leitura) e o ``Action Block`` (processamento). O recurso ``propagate completion`` garante que o fluxo só avance para a próxima etapa quando a anterior tiver sido concluída, o que é um aspecto muito interessante para o controle do fluxo.

- * ****Uso de Memória com Span para Desempenho****: Para otimizar o uso de memória em cenários de alto desempenho e reduzir a pressão sobre o Garbage Collector, a sugestão foi utilizar o `Span<T>`. O `Span<T>` permite manipular pequenas porções contíguas de memória, alocadas na stack, para processar grandes arquivos ou volumes de dados sem carregar o arquivo inteiro na memória.
- * ****Memory e Comparativo com Span****: O `Memory<T>` é semelhante ao `Span<T>`, mas pode ser armazenado na heap, o que o torna utilizável em operações assíncronas (`async/await`) e como campo de classe, ao contrário do `Span<T>`. A manipulação de grandes fluxos de dados de rede por meio de buffers e slices é um exemplo de seu uso eficiente, minimizando realocações de memória.
- * ****Array Pool para Reutilização de Arrays****: O conceito de `Array Pool` foi introduzido como um "pool de arrays" que armazena arrays de diferentes tamanhos para reutilização, evitando alocações desnecessárias e reduzindo a pressão do Garbage Collector. A alocação contígua de arrays na memória por questões de hardware foi explicada, destacando que a criação de um array maior requer realocação e cópia de dados, o que é custoso.
- * ****ValueTask para Otimização de Assincronismo****: A `ValueTask` é uma alternativa à `Task` que evita a criação da máquina de estados e toda a estrutura de assincronismo quando o valor de retorno já está pronto ou "quente" (como em um cache hit). Seu uso é recomendado em casos onde se espera que, na maioria das vezes, a operação assíncrona já tenha um resultado imediato, eliminando o overhead de alocação da `Task`.
- * ****IAsyncEnumerable para Leitura de Dados em Partes****: O `IAsyncEnumerable` permite a execução postergada de operações assíncronas, como a leitura de dados de uma API em partes. Essa abordagem é ideal para cenários onde os dados chegam ao longo do tempo (como em uma rede), pois melhora a eficiência e a capacidade de resposta da aplicação, processando os dados à medida que são recebidos.
- * ****Uso de Coleções Concorrentes (ConcurrentDictionary)****: Para lidar com o acesso simultâneo de múltiplas threads a variáveis e estruturas de dados, deve-se usar as coleções concorrentes do .NET, como o `ConcurrentDictionary`. Estas coleções implementam mecanismos de bloqueio internamente, de forma transparente, permitindo

acesso simultâneo sem a necessidade de implementar bloqueios explícitos (`lock`)\~=CITATION:127=\~\~=CITATION:129=\~.

- * ****Boas Práticas de Integração, Logging e Timeout****: É essencial prever falhas em integrações, implementando mecanismos de resiliência, como o uso de blocos `try/catch` e estratégias de retentativa, como o Polly, para garantir que a aplicação não pare de funcionar. A gestão de timeouts é muito importante para evitar que a aplicação sobrecarregue outros sistemas, e o monitoramento e logging detalhado são fundamentais para identificar problemas precocemente\~=CITATION:131=\~.
- * ****Recomendação de Literatura Técnica****: Fábio Diniz recomendou o livro "Pro C\#\# 7: With .NET and .NET Core" (ou versões posteriores) para entender a evolução do assincronismo no .NET e "Sistemas Operacionais Modernos" para compreender o funcionamento do hardware e do sistema operacional\~=CITATION:133=\~. A compreensão do funcionamento interno do hardware e do SO ajuda o desenvolvedor a tomar melhores decisões na programação\~=CITATION:135=\~.
- * ****Obsessão por Tipos Primitivos (Value Objects)****: Foi fortemente sugerida a criação de tipos para conceitos importantes (como CPF, CNPJ, Email) em vez de usar strings ou inteiros em todo o código, seguindo o conceito de value objects\~=CITATION:136=\~. Isso aumenta a expressividade do código e permite que a lógica de validação (como a do CPF) fique encapsulada dentro do tipo, reduzindo o esforço do Garbage Collector e proporcionando ganhos de desempenho\~=CITATION:137=\~\~=CITATION:139=\~.

Revise as anotações do Gemini para checar se estão corretas. [Confira dicas e saiba como o Gemini faz anotações](#)

Como está a qualidade de **destas observações**? [Responda a uma breve pesquisa](#) para nos dar seu feedback, incluindo o quanto as observações foram úteis para o que você precisa.